Progress® Agentic RAG

The Agentic Knowledge Layer: Building One Infrastructure for Every AI Use Case

WHITEPAPER

Table of Contents

Introduction

As your organization continues to expand its AI initiatives and create more AI-powered experiences, you've probably noticed that teams are repeating the same work. New use cases require separate ingestion pipelines, new vector stores, new retrieval configurations and another set of controls, written from scratch each time. As a result, operational expenses and effort grow with every new AI deployment—that's a lot of duplicate work and wasted time, every time.

The Agentic Knowledge Layer is the architectural exit from that cycle. Instead of creating a new pipeline for every use case, you'll have a single indexed knowledge foundation with multi-layer retrieval across semantic vectors, full text and knowledge graphs, configurable per application without re-architecting the underlying stack.

With the Agentic Knowledge Layer, governance, access controls and source attribution are built into the retrieval layer once and inherited by every AI experience built on top of it, allowing you to swap your large learning models (LLM) without touching your knowledge infrastructure. As a result, adding a new use case takes days, instead of months, because the foundation already exists.

In this whitepaper, we take a deep dive into the Agentic Knowledge Layer.

What Is the Agentic Knowledge Layer?

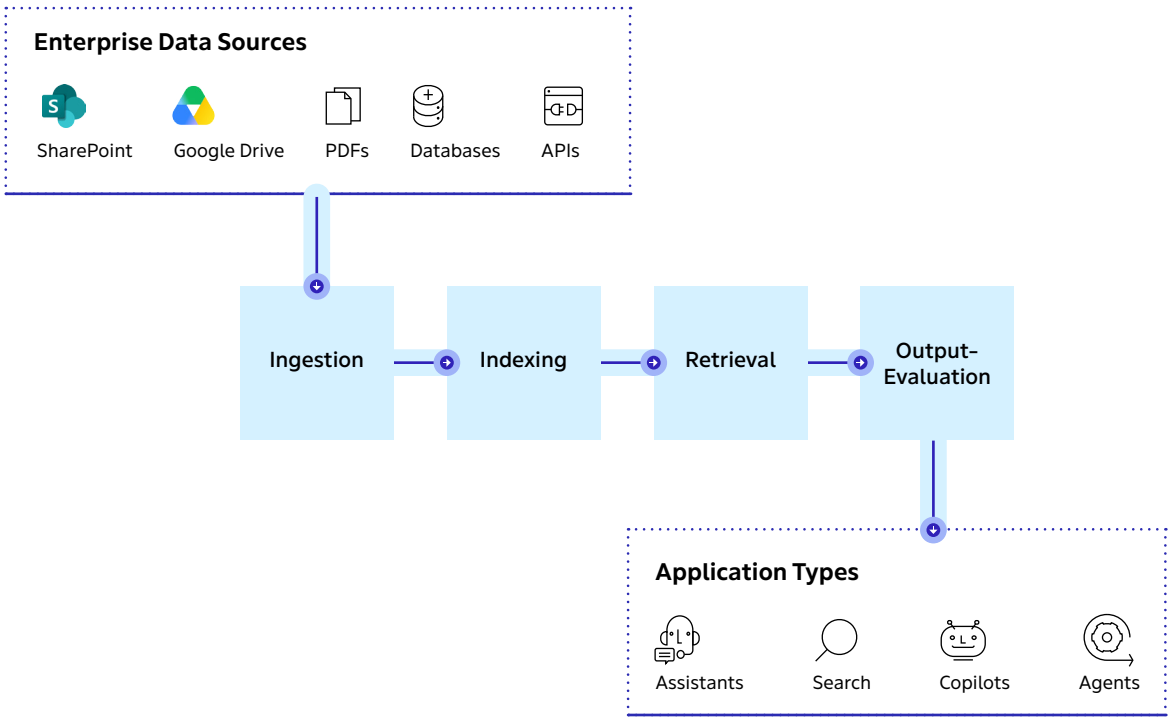
The Agentic Knowledge Layer is a managed infrastructure layer that sits between your enterprise data and every AI experience built on top of it. It handles ingestion, indexing, retrieval, evaluation and governance as a shared, ongoing service, rather than a per-application build.

Whereas a retrieval-augmented generation (RAG) pipeline is something you construct and maintain for a specific application, the Agentic Knowledge Layer can be deployed only once and configured per use case. The distinction matters architecturally: applications consume a shared knowledge API, making the knowledge infrastructure a durable platform asset that compounds in value with each use case you add.

The Agentic Knowledge Layer is comprised of four functional components that work in sequence:

1. **Smart ingestion layer** that transforms raw enterprise content into searchable knowledge
2. **Multilayer index** that makes that knowledge retrievable across semantic, full-text and graph dimensions simultaneously
3. **Retrieval agent layer**, which dynamically selects and orchestrates the right retrieval strategy per query
4. **Built-in evaluation framework** that continuously scores output quality against measurable criteria called **REMi** (RAG Evaluation Metrics Intelligence)

Each component is modular, configurable and decoupled from any specific LLM, which means the knowledge infrastructure you build today does not become a liability when the model landscape evolves over time.



The Knowledge Box: Your Indexed Enterprise Foundation

The primary container in the Agentic Knowledge Layer, the **Knowledge Box**, is a governed, indexed store that holds all content ingested from your systems and exposes it as a queryable knowledge surface. It's not simply a database, but a living, continuously updated knowledge base. It combines document content with the relationships between entities, the metadata that describes each resource and the access controls that determine who can access what information.

A single Knowledge Box can store documents, images, videos, audio files and web pages with enriched metadata that makes them discoverable at query time, not just searchable by file name. Multiple AI experiences can query the same Knowledge Box at the same time, each applying different retrieval configurations, without affecting the underlying data store.

Smart Ingestion: From Unstructured Content to Discoverable Knowledge

Ingestion in the Agentic Knowledge Layer converts unstructured enterprise content into structured, enriched, retrieval-ready knowledge.

When a document enters the system, it passes through multiple processes:

- **Semantic chunking** that preserves contextual coherence across passages
- **Entity extraction** that surfaces the people, places, organizations and concepts within it
- **Metadata enrichment** that makes it filterable and rankable at query time
- **Access control tagging** that facilitates downstream retrieval and respects the permissions the organization has defined

This pipeline runs continuously. **Sync Agents** monitor connected data sources, such as Microsoft SharePoint, Google Drive, Dropbox, Progress® ShareFile® software, local folders and other web sources, and automatically ingest new and updated content without manual intervention. This way, the knowledge layer is always aligned with up-to-date enterprise knowledge. What the knowledge layer knows today reflects what the organization knows today, not what was indexed at the last scheduled job.

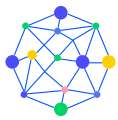
Multilayer Indexing: Why Single-Strategy Retrieval Fails in Production

Most RAG implementations index content once using a single embedding strategy. Retrieval usually leverages semantic vector search. That works well for general-purpose queries, but breaks down reliably for the queries that matter most in production—like precise terminology lookups, entity-specific questions or queries that require connecting information distributed across multiple documents.

The Agentic Knowledge Layer solves this problem by indexing across three layers simultaneously:

Semantic vector search retrieves content based on meaning rather than exact terms. It makes the right choice for nuanced, context-dependent queries where the user's intent matters more than their specific vocabulary. It can even handle different types of vector representations.

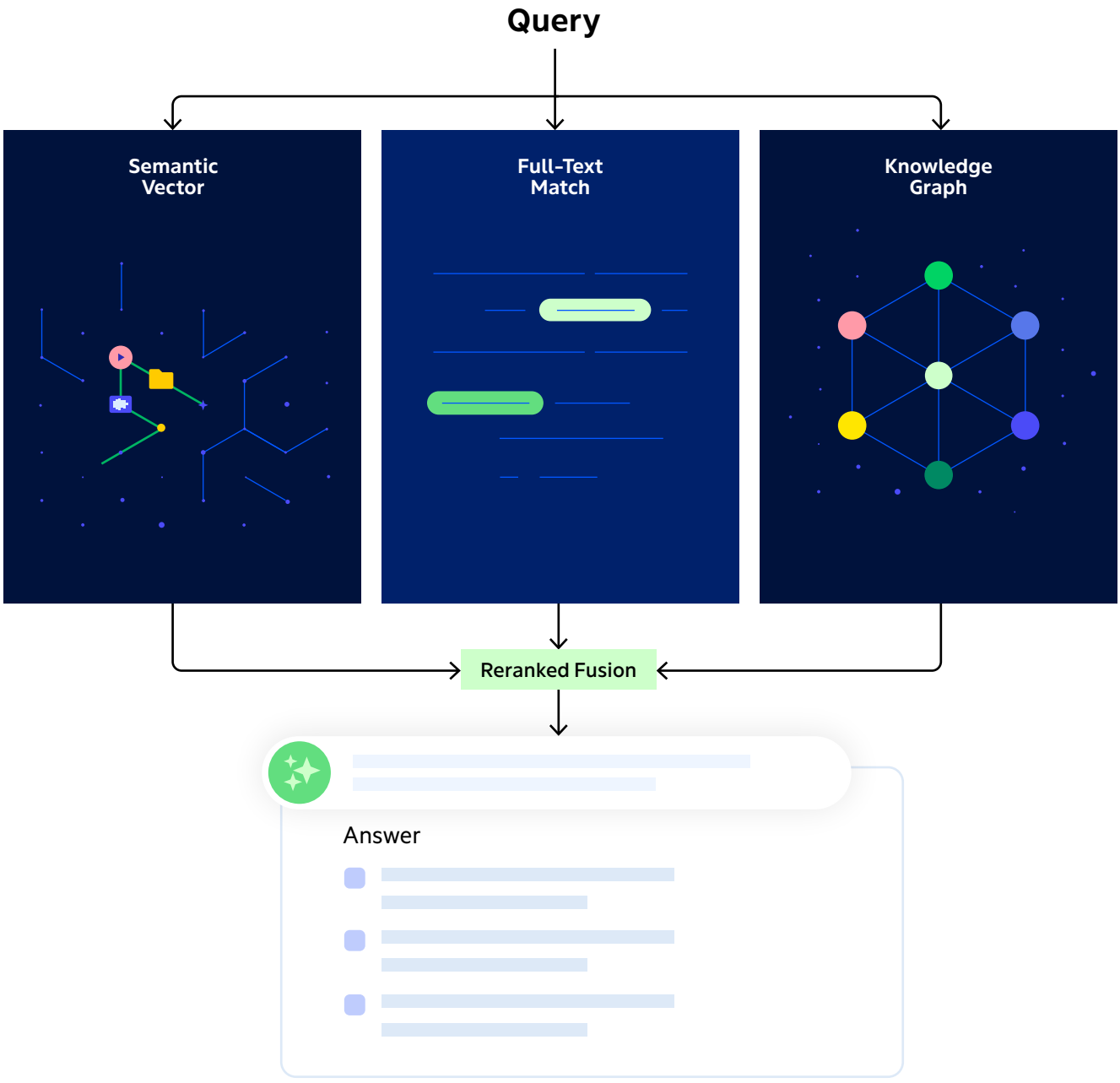
Full-text search retrieves based on keyword precision, which is essential for queries involving specific product names, regulatory citations, error codes or any terminology where exact match matters.



Knowledge-graph search traverses the entity graph to surface indirect relationships. This functions as the layer that answers questions such as, *"What contracts involve this counterparty and this jurisdiction?"* It follows connections rather than matching strings.

These layers are not applied sequentially. Instead, they are queried in combination, with the system weighting results across all three based on the retrieval strategy configured for the application. In fact, the same query may return very different results depending on the application. For example, a compliance assistant restricted to policy documents will surface different information than an engineering copilot searching across technical specifications.

The result is a retrieval system that can adapt to the needs of each application, rather than forcing every query through the same search strategy.



LLM-Agnostic by Design: Decoupling Knowledge Infrastructure from Model Decisions

Hardcoding a large language model (LLM) into your application logic is one of the most common and costly architectural mistakes in early RAG implementations. We understand that it feels like a reasonable shortcut: pick the best model available, build around it and ship faster. However, problems surface later when the model is deprecated or when pricing changes make it uneconomical. Or, maybe a new model delivers meaningfully better performance at lower cost, but you can't take advantage of it without rebuilding the underlying retrieval layer.

Model deprecation is a real problem—37% of organizations now run five or more models in production. What's more, the churn of model releases, pricing adjustments and provider policy changes has made model flexibility a core infrastructure requirement.

The Agentic Knowledge Layer is fully LLM-agnostic. The knowledge infrastructure (ingestion, indexing, retrieval and evaluation) is entirely decoupled from the generation layer. Thus, the Progress® Agentic RAG solution supports OpenAI's GPT, Anthropic's Claude, Mistral, Google Gemini and other models interchangeably.

This means that the Knowledge Box you built for GPT-4o works identically with Claude 3.5 or Mistral Large the day you decide to switch—with no changes occurring to the underlying data pipeline. Bring-your-own-key support means your model relationships, vendor contracts and spending stay under your control, the platform does not intermediate your model access or mark up your inference costs.

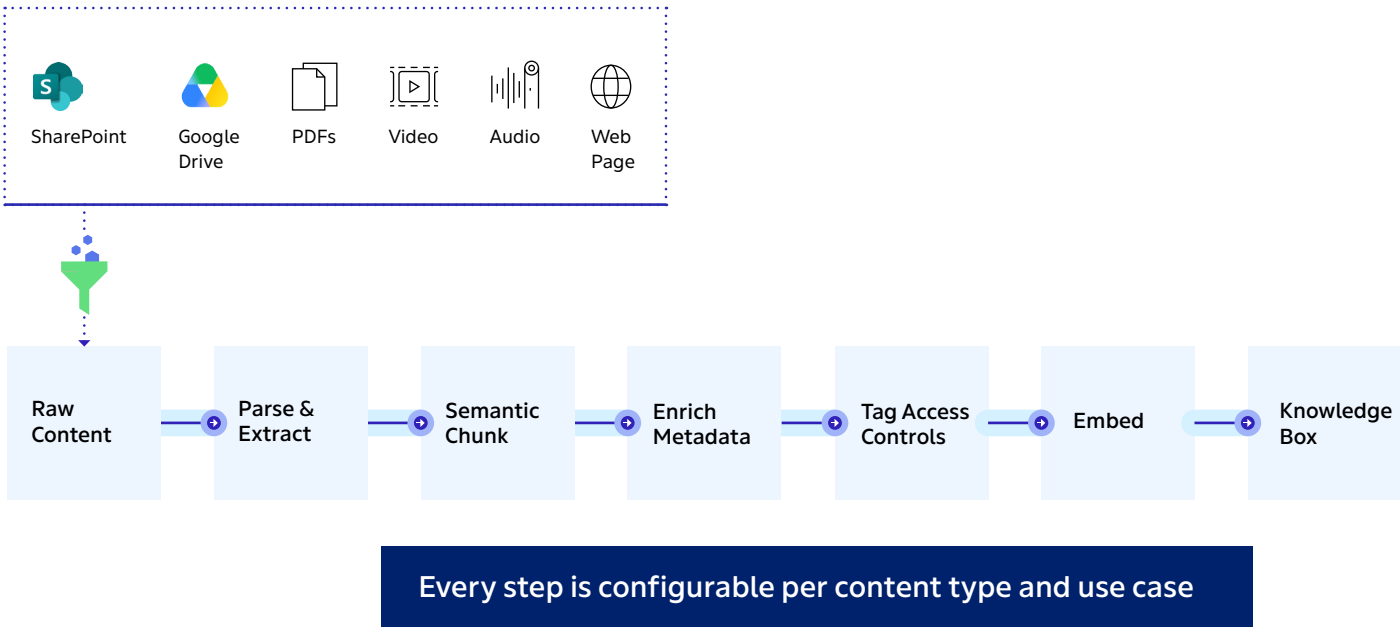
This separation turns the knowledge layer into a reusable platform, rather than a one-off project. In many RAG architectures, the LLM is tightly coupled to the rest of the system, making model changes disruptive and expensive. The Agentic Knowledge Layer separates the knowledge infrastructure from the model, allowing organizations to adopt new LLMs without the need to retrain, reindex or rebuild their retrieval, governance or knowledge management foundation.

Ingestion: Building a Knowledge Foundation for Enterprise Content

Most enterprise data pipelines treat ingestion as a simple loading exercise: they load content into the system and move on. However, in production RAG, that framing is the root cause of most retrieval failures. As a result, decisions made at this stage—i.e., what content is indexed, how it is chunked, what metadata is attached or which access controls are applied directly—affect how well retrieval works later. Poor ingestion practices propagate through the entire pipeline, reducing retrieval accuracy and degrading the quality of generated answers. In other words, garbage in . . . garbage out.

By contrast, the Agentic Knowledge Layer treats ingestion as a transformation pipeline. Raw enterprise content enters as unstructured source material and exits as structured, enriched, access-controlled, retrieval-ready knowledge. The difference between those two states is the work the ingestion layer does, and it is significant.

Ingestion Pipeline



Every step is configurable per content type and use case

What Gets Ingested

The Agentic Knowledge Layer is built to handle the reality of enterprise content: heterogeneous, distributed and predominantly unstructured. Supported content types include documents (PDF, Word, HTML, etc.), images, video and audio all processed into queryable knowledge, regardless of source format.

Content can be ingested programmatically via a REST API, Python SDK or JavaScript SDK, then uploaded manually through the dashboard or synchronized automatically from connected cloud storage.

The method of ingestion is also important. A common production mistake is indexing everything including page headers, navigation elements, boilerplate footers, and other structural noise that pollutes the index and degrades retrieval precision. The ingestion layer provides controls to scope exactly what gets indexed from each source, so the Knowledge Box reflects the content that should be retrievable, not the full byte stream of every document.

Semantic Chunking: Preserving Context Across Passages

How content is divided into chunks at ingestion directly affects what the retrieval layer is able to surface. While fixed-size chunking is fast and predictable, it also cuts across sentences, severs tables and splits arguments mid-paragraph. This results in content fragments that lack the context needed to understand them.

For production workloads in compliance, legal, technical and policy-heavy domains, semantic chunking is now considered the minimum viable approach.

However, the Agentic Knowledge Layer uses configurable split strategies that respect semantic boundaries. This more holistic approach keeps logically coherent passages intact, honors headers and section structures and preserves the contextual relationship between a claim and the evidence that supports it. Also, split strategies are reusable and configurable per content type. For example, the chunking logic applied to a legal contract would differ from the logic applied to a product spec, without requiring separate pipelines.

Metadata Enrichment and Entity Extraction

Retrieval quality at query time depends entirely on the quality of the metadata attached during ingestion. The Agentic Knowledge Layer enriches every resource with two classes of metadata: 1) **system metadata** extracted automatically (file type, language, creation date, source system, etc.); and 2) **user-defined metadata** that teams configure to reflect their organizational taxonomy: tags, classifications, sensitivity levels, department labels and content types.

Beyond metadata, the ingestion layer runs **graph extraction** to identify named entities such as people, organizations, locations or concepts and the relationships between them. This information feeds the knowledge graph that powers relationship-aware retrieval. For example, when a document mentions a contract counterparty, a jurisdiction and a compliance requirement, those elements are captured as connected entities during ingestion. This allows the graph retrieval layer to follow the relationships between them when answering complex queries.

Ingestion agents can automatically classify, summarize and extract information from documents, creating a structured representation that improves retrieval and answer quality.

Sync Agents: Continuous Ingestion Without Manual Intervention

One of the most common failure modes in enterprise RAG is knowledge staleness. This is when the agent draws on content that no longer reflects the current state of the organization. In fast-moving domains such as compliance, pricing, product documentation or support knowledge, stale retrieval is an unacceptable tradeoff.

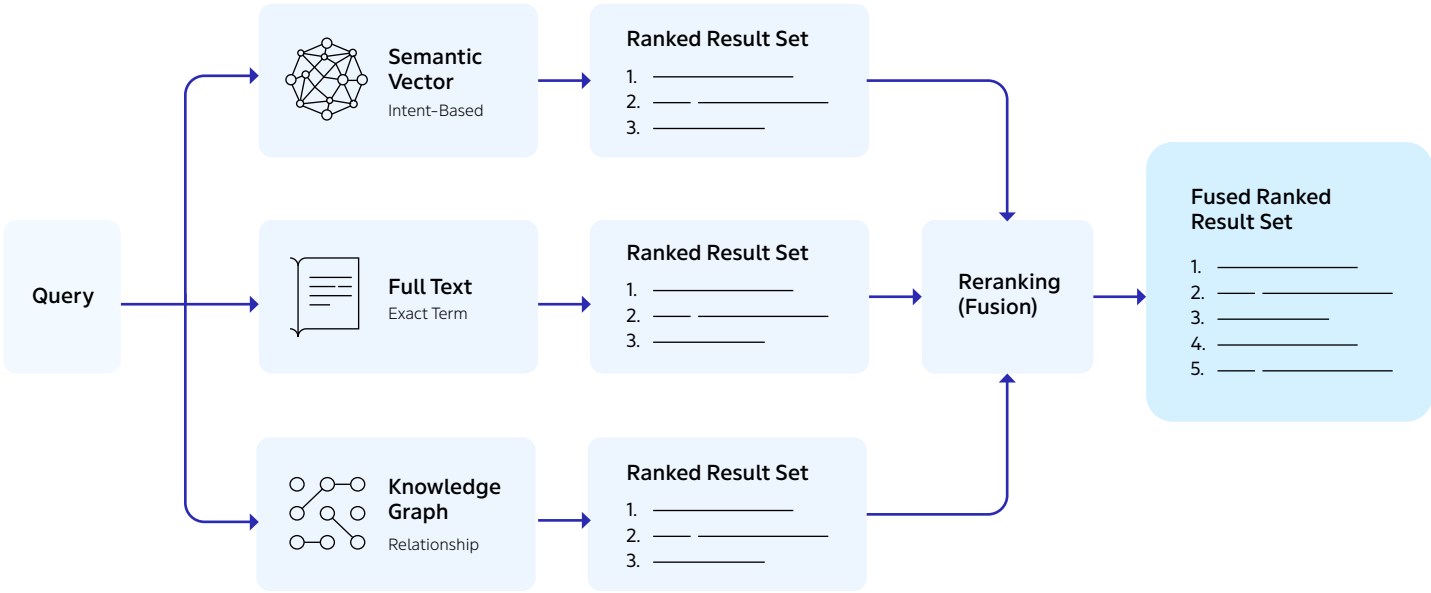
Sync Agents and **Sync Service** work to eliminate this failure mode by continuously monitoring connected data sources and automatically ingesting new and updated content as it appears. Supported integrations for Sync Agents include Microsoft SharePoint, Google Drive, Progress ShareFile, Dropbox. While

the Sync Service supports local folders and other web sources via sitemap. The Sync Service can be deployed locally or on a server, and run via Windows, macOS and Linux, making them a seamless fit into existing infrastructure without the need for a dedicated ingestion service.

For teams that need programmatic control over ingestion cadence, the REST API and SDKs support fully automated, scheduled ingestion workflows. Whether you choose Sync Agents or API-driven ingestion, it is a configuration decision, not an architectural constraint since both write to the same Knowledge Box and produce the same enriched, indexed output.

Multilayer Indexing: Why Single-Strategy Retrieval Fails at Scale

The most common retrieval architecture in production RAG is also the most common source of production failures: a single vector index, queried with semantic search, expected to handle every query type the application receives. While this approach performs well in demos, it often breaks down in production. For example, queries that require precise matches, such as regulatory citations, contract obligations or error codes, are where semantic similarity *alone* is least effective.



Across enterprise deployments, many RAG implementations fail to reach production, and retrieval quality is consistently among the primary causes.

The Agentic Knowledge Layer indexes across three layers simultaneously and queries all three during retrieval. Then the results are combined into a single ranked output. No single retrieval strategy works for all queries. For instance, semantic search finds meaning, but misses precision, and keyword search finds exact terms, but misses intent. Graph traversal finds relationships but misses unconnected content. Each layer serves a different purpose—but together, they provide the range of retrieval capabilities needed for enterprise applications.

Layer 1: Semantic Vector Search—Retrieval by Meaning

Semantic search operates on embedding space. Every chunk in the Knowledge Box is encoded as a high-dimensional vector representing its meaning, and queries are matched by proximity, cosine similarity or dot product depending on the configured embedding model. The result is retrieval that surfaces contextually relevant content even when the query and document share no exact vocabulary, and handles multilingual queries by the same principle: meaning is preserved across languages in embedding space, even when surface terms differ.

The limitation is precision. Semantic search scores similarity, and similarity is not the same as correctness. For queries that require an exact match (e.g., a product SKU, a regulatory code or named entity), the highest-scoring semantic result is not necessarily the right one, and a different retrieval layer is required for those cases.

Layer 2: Full-Text Search—Retrieval by Precision

Full-text search uses BM25, a keyword-based ranking function that scores documents by term frequency and document-length normalization. Compliance documents, legal contracts, product catalogs, error logs and technical specifications are built on precise terminology—for example, a query for “Section 12.4(b)” or a specific CVE identifier is a precision question. Thus,

semantic search will either miss it entirely or answer with a confident adjacent result that happens to be wrong. Full-text search retrieves the document that contains *exactly* the desired terms.

The scoring mechanisms for full-text and semantic search are fundamentally incompatible; BM25 scores and embedding distances don't live on the same scale. The Agentic Knowledge Layer fuses results by re-ranking and normalizing across retrieval modes, producing a unified ranked list. This way, teams don't need to build and maintain their own score-normalization logic.

Layer 3: Knowledge-Graph Search—Retrieval by Relationship

Vector and full-text search can retrieve relevant documents, but they cannot reason over the relationships between entities or trace connections across multiple documents.

Knowledge-graph search works on the network of entities created during ingestion. This makes it possible to answer questions that depend on how people, organizations, documents and other entities are connected. For example, finding contracts that involve a specific counterparty in a specific jurisdiction requires following those relationships across the data, which is something that traditional methods can't do reliably.

Unlike semantic search, graph search can follow relationships across documents to connect related information and works best for questions about how specific entities are related to one another.

And the three layers are complementary by design: no single layer covers the full query surface, and the combination is what makes retrieval production-grade.

Fusion and Reranking: Producing a Single Ranked Output

While combining retrieval strategies can produce the best results, running three retrieval strategies in parallel produces three ranked result lists with incompatible scoring systems. The Agentic Knowledge Layer fuses these result lists internally, applying configurable weighting across retrieval modes. In this way, teams can tune the balance for their specific application context.

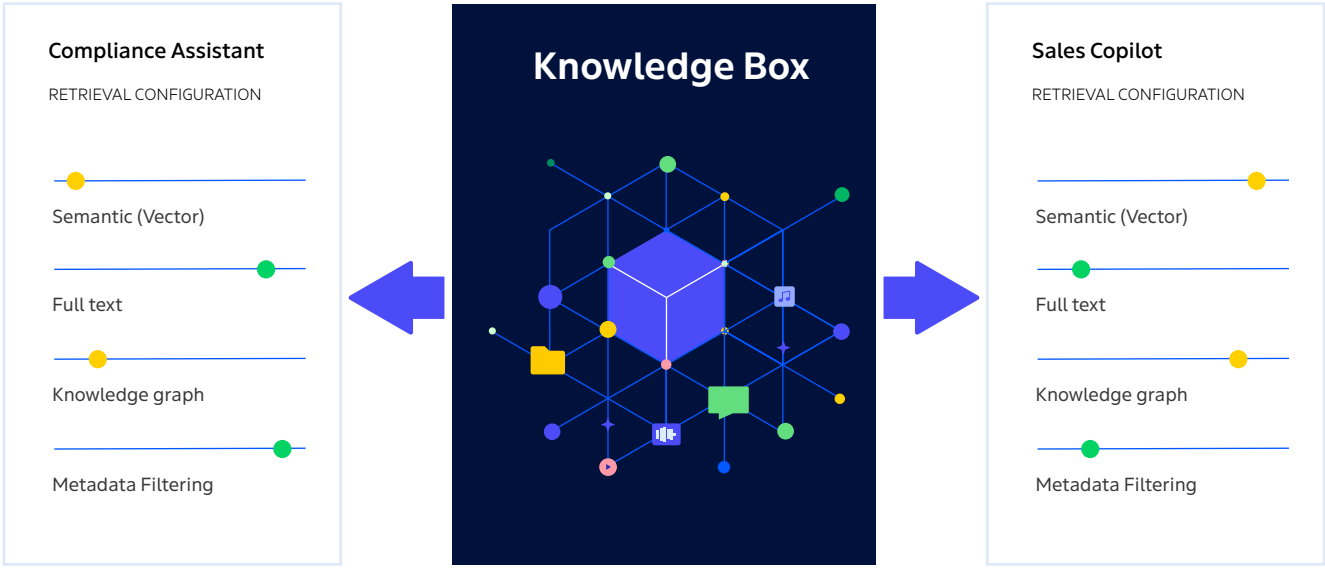
For example, a compliance assistant can weigh full-text and graph results more heavily, while a conversational assistant or sales copilot can lean on semantics using the same Knowledge Box. Teams can reconfigure the retrieval strategy per application, without modifying or destabilizing the underlying index. This flexibility enables you to optimize retrieval for each application or use case without duplicating data, rebuilding indexes or creating separate knowledge repositories.

Retrieval as Configuration, Not Reconstruction

In a pipeline-per-use-case architecture, every new AI application comes with a retrieval problem to solve from scratch. What data sources are relevant? What chunking logic is appropriate? Should the application weight keyword precision over semantic relevance, or the reverse? Should results be filtered by department, sensitivity level or document type?

Each of these decisions is embedded into the application's retrieval code, and each time the answer changes (due to a new data source, use case or ranking requirement) you have to change the code as well. Retrieval becomes a continuous reconstruction project, rather than a configurable layer.

With the Agentic Knowledge Layer, retrieval behavior is exposed as configuration: a named set of parameters that defines how a search should be performed, applied at query time against the shared index and adjustable without touching the underlying data pipeline.



Same index. Different retrieval configuration.
No separate pipelines.

But building a shared knowledge foundation is only part of the challenge. Next, we examine how the Agentic Knowledge Layer transforms retrieval from a fixed process into a configurable capability.

Search Configurations: Stored, Reusable Retrieval Parameters

A search configuration is a reusable definition of how a query should be executed. It stores retrieval strategy selection, filter expressions, security context, ranking logic and prompt parameters and references them by name at query time. This allows you to define a configuration once, apply it across as many queries as needed and update it centrally when retrieval behavior needs to change.

At an operational level, this approach separates the retrieval definition from the application code. An application doesn't embed retrieval logic, but instead calls a named configuration. When that configuration needs to be tuned, it is updated in one place and the improvement propagates immediately across every application using it, without the need to deploy code.

Use-Case Differentiation Without Data Duplication

In a traditional RAG architecture, each use case is a separate build. In the Agentic Knowledge Layer, each is a retrieval configuration adjusting chunking strategy, ranking logic, search weighting between keyword and semantic modes, metadata filtering, and model orchestration parameters. More than 30 retrieval strategies are available, ranging from highly targeted searches focused on precision to broad, multisource queries that support agentic workflows. This allows sales, engineering, HR, compliance and operations teams to use AI applications tailored to their specific needs while relying on the same shared knowledge foundation.

Dynamic Retrieval: Agentic Query Orchestration

Static retrieval configurations handle the majority of production query patterns. But for more complex agentic workloads, like multistep reasoning tasks where the right data source depends on what was retrieved in the previous step, the Retrieval Agent layer adds dynamic orchestration on top. A Retrieval Agent can analyze an incoming question, break it into smaller subquestions and evaluate the information collected at each step. It can then iterate as needed, pulling information from multiple Knowledge Boxes, SQL databases, internet search services or MCP servers until it has enough context to generate a complete answer.

Tuning Without Rebuilding

Retrieval quality is not a fixed property of a knowledge layer, it improves over time as teams observe how queries perform, identify ranking or relevancy gaps and adjust configuration to address them. The Agentic Knowledge Layer supports iterative retrieval refinement without touching the underlying index or destabilizing other applications consuming the same Knowledge Box.

This is the operational property that makes the knowledge layer a platform rather than a project. A bespoke RAG pipeline requires careful, coordinated changes to adjust retrieval behavior, impacting ingestion code, index configuration and application logic simultaneously. A configured retrieval layer separates those concerns cleanly: the index is stable, the retrieval configuration is tunable and improvements compound across every use case.

Quality and Governance Built into the Retrieval Layer

Retrieval quality and governance determine whether an enterprise AI deployment reaches production, but they are almost always treated as separate concerns and addressed by different teams at different stages of the build. Evaluation gets added after the application is assembled, typically as manual spot-checking that works at low query volumes and collapses as usage scales. Governance gets bolted on downstream, applied to outputs after the retrieval layer has already surfaced them. Both approaches fail in the same way: they create a gap between what the system retrieves and what the organization can defend.

The Agentic Knowledge Layer addresses both processes during retrieval. The system continuously evaluates quality using real-world queries instead of manual spot checks. It also enforces governance at retrieval time, enabling access controls, attribution and auditability to be built into every response from the start.

REMi: Continuous, Measurable Retrieval Quality

Most teams evaluate RAG systems through manual testing. They review a sample of queries, spot-check the answers and conclude that the system works well enough, until users start reporting problems.

This approach does not scale to production because it misses rare, but important, query patterns and fails to identify gradual declines in retrieval quality. Worse yet, it provides little insight into whether an issue stems from retrieval, generation or the underlying data.

REMi is the Agentic Knowledge Layer's built-in evaluation framework, running continuously against live query traffic. It assesses output quality across three dimensions simultaneously:

- **Answer relevance:** Does the response actually address what the user asked?
- **Context relevance:** Were the retrieved documents genuinely relevant to the query?
- **Groundedness:** Is the response supported by the retrieved content, or is the model introducing claims not present in the source material?

Critically, REMi diagnoses where quality problems originate. A low groundedness score points to a generation problem. A low context relevance score points to a retrieval configuration problem. A low answer relevance score points to a data gap in the Knowledge Box. Teams get specific, actionable signals rather than a binary pass/fail, and they get it continuously, before users encounter a failure.

Governance at the Point of Retrieval

A significant share of enterprise AI pilots fail to reach production for a reason that has nothing to do with model performance: teams often manage security and access controls at the application layer, applying them inconsistently across deployments and rebuilding them from scratch for every new AI agent, often leading to governance gaps and increased operational overhead.

The Agentic Knowledge Layer enforces governance at the retrieval layer, where it belongs. Role-based access control (RBAC) enables a user to retrieve only content they are authorized to access in the source system, consistently applied across every AI experience that queries the Knowledge Box.

Additionally, tenant isolation prevents data from crossing between users or contexts. Every response carries source attribution traceable to the original document or data point, making outputs auditable and defensible rather than opaque.

The entire substrate ships with SOC 2 Type II, ISO 27001 and GDPR certifications, encrypted at rest with AES-256 and in transit with TLS, and subject to regular third-party security audits. These are not features layered on top of the retrieval layer; they are properties of the retrieval layer, inherited by every application built on top of it.

Deployment and Integration

Deployment is a key architectural decision in enterprise AI because it determines where data resides, how compliance requirements are met and how much operational control teams retain.

The binary choice between “cloud” and “on-premises” that characterized earlier enterprise software generations has become a spectrum between SaaS, hybrid, Virtual Private Cloud (VPC)-isolated, fully self-hosted and air-gapped deployments. Each carries distinct trade-offs between agility, control and cost, but it’s the knowledge layer underneath your AI experience that needs to fit your posture rather than dictate it.

The Agentic Knowledge Layer is available across all three primary deployment models (SaaS, hybrid and fully on-premises), leaving the deployment choice to the customer. The same knowledge infrastructure, retrieval capabilities, governance controls and evaluation framework operate consistently regardless of where the layer is deployed.

The following sections explore the options available for bringing the Agentic Knowledge Layer into production at scale.

SaaS: Fastest Path to a Production Knowledge Layer

The hosted SaaS deployment model is the fastest path from evaluation to production for which Progress handles provisioning, infrastructure management, scaling and uptime, allowing engineering to focus on building AI experiences rather than maintaining the knowledge layers beneath. SaaS deployment also includes a comprehensive compliance framework, including SOC 2 Type II, ISO 27001 and GDPR support.

If cloud deployment is straightforward and your priority is time to value, SaaS is the right starting choice for your organization. Progress offers a 14-day free trial on the hosted deployment, which means the evaluation environment is identical to the production environment.

Hybrid and On-Premises: Deployment Without Compromise

If you have data residency requirements, regulatory obligations or internal policies that limit the use of public cloud, you can deploy the Agentic Knowledge Layer in a private cloud or fully on-premises environment.

These deployment options provide the same capabilities as the hosted offering, including multilayer indexing, 30+ retrieval strategies, the REMi evaluation framework and built-in RBAC and governance controls. Regardless of where it is deployed, the knowledge layer delivers the same functionality and user experience.

The API and SDK Surface: Integration Without Lock-In

The Agentic Knowledge Layer exposes its full capability surface through open APIs and SDKs designed to integrate into existing engineering workflows rather than requiring a build around a proprietary interface.

Four integration paths are available:

- **REST API:** Covers the complete surface of ingestion, retrieval, search configuration, Knowledge Box management and evaluation. Language-agnostic and suitable for any backend architecture.
- **Python SDK:** Wraps the REST API for Python environments, with additional support for popular web frameworks including Django. Includes a Command Line Interface (CLI) for terminal-based operations, scripted ingestion jobs and local development workflows.
- **JavaScript/TypeScript SDK:** Integrates directly into front-end application code and modern JavaScript frameworks, including React, Next.js, Angular, Vue.js and Svelte. The right integration path for application developers building AI experiences at the UI layer.
- **Dashboard:** Designed for manual operations, Sync Agent configuration, Knowledge Box management and REMi monitoring, this web UI is useful for content teams, operations staff and engineers who need visibility into the knowledge layer without writing API calls.

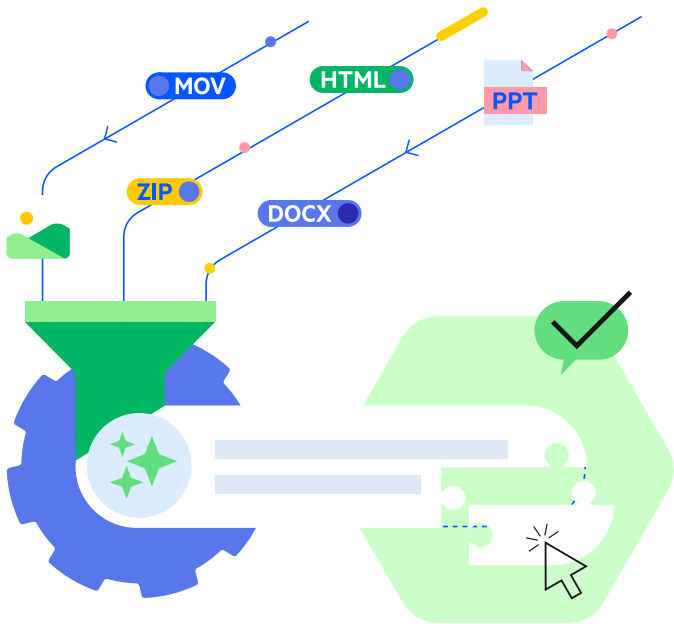
All platform capabilities are exposed through the REST API, with SDKs providing streamlined access for common development environments. Because new features are introduced at the API layer first, teams can always integrate directly to access the latest functionality.

RAG-as-a-Service: Consuming the Knowledge Layer as an API

The Agentic Knowledge Layer is designed to enable RAG-as-a-Service, where the knowledge infrastructure is deployed and maintained as a shared service and application teams consume it through the API, rather than owning or operating any part of the retrieval stack. A compliance assistant, a sales copilot and an internal search tool all consume the API, sending queries, receiving grounded results with source attribution and returning answers to users.

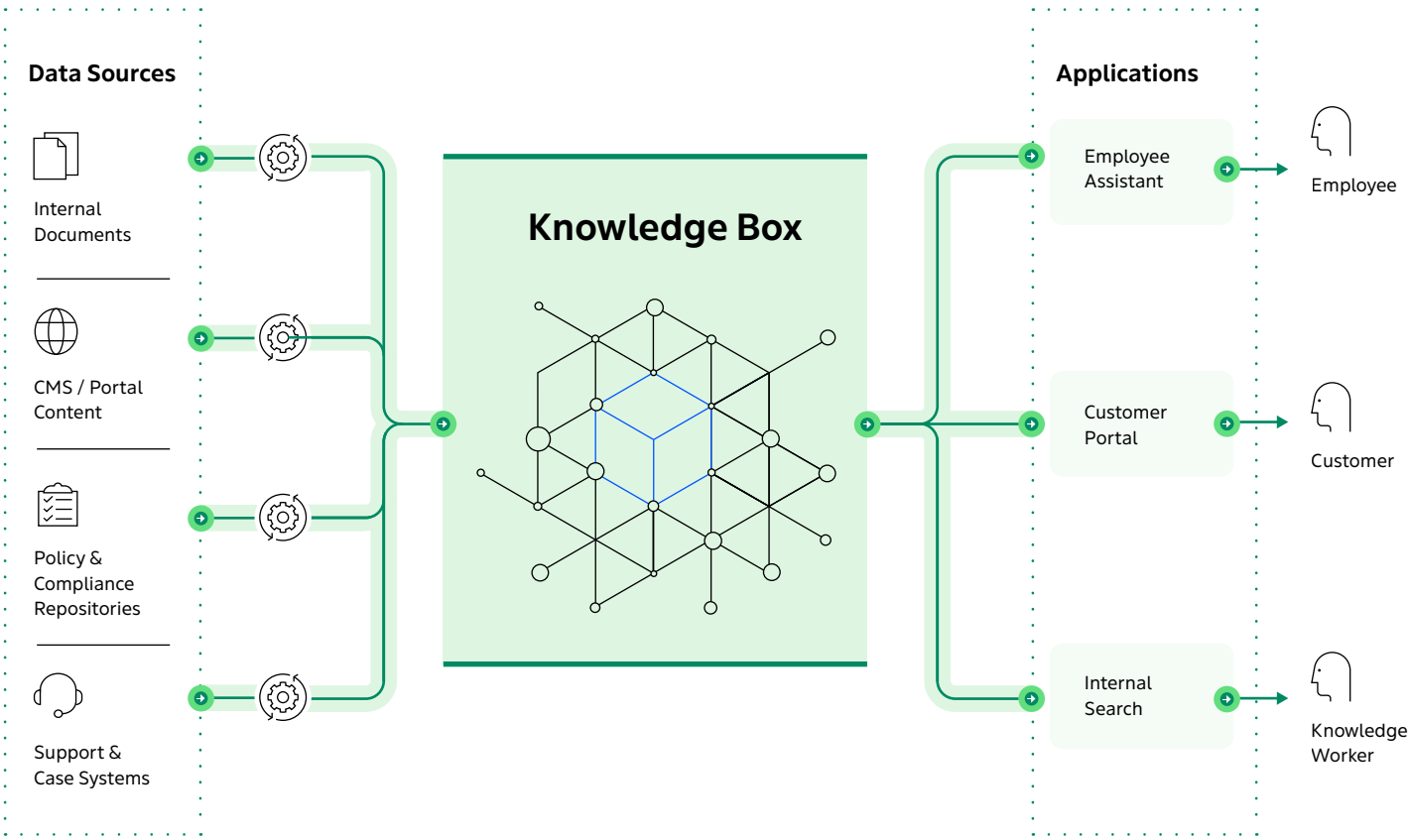
Importantly, none of these applications manage ingestion pipelines, maintain indexes, tune retrieval logic or operate evaluation tools or governance controls because the knowledge infrastructure lifecycle is decoupled from the application lifecycle. Instead, teams manage, update and improve the knowledge layer centrally and every application benefits from those changes automatically. So when they deploy a new AI use case, they build on an existing knowledge foundation.

The same knowledge layer can support custom applications including Frontier, Agentforce, Gemini and other orchestration frameworks simultaneously. The knowledge layer remains consistent while teams configure everything built on top of it.



One Layer, Many Experiences: A Reference Architecture for Real-World Production Scenarios

Here is a single Progress Agentic RAG deployment powering three distinct AI experiences and running simultaneously from the same indexed Knowledge Box: an employee-facing assistant, a customer self-service portal and an internal search tool—with no duplicate pipelines or shared retrieval logic beyond the layer they all consume.



GOVERNANCE & EVALUATION LAYER (SHARED ACCROSS ALL APPLICATIONS)

RBAC (Role-Based Access Control) | REMi (Evaluation & Monitoring)

Experience 1: Employee-Facing AI Assistant

The employee assistant typically serves the broadest range of queries in an enterprise environment, covering topics such as HR policies, IT troubleshooting, product knowledge and internal procedures. To support this diversity of use cases, its retrieval configuration emphasizes broad semantic coverage while prioritizing the most current content.

RBAC and content labeling restrict retrieval to information that *each* employee is authorized to view. And when teams add new HR policies or update product documentation, the employee assistant automatically incorporates that content from the Knowledge Box, assisting in employees receiving the most current information.

The same Knowledge Box that serves this assistant also serves the other three experiences in this architecture. Nothing about the employee assistant's retrieval configuration affects the retrieval behavior of the compliance feed or the customer portal; each application operates against shared indexed content through its own named search configuration.

Experience 2: Customer Self-Service Portal

The customer portal is the highest-stakes external-facing experience in this architecture. Answers surface directly to customers, accuracy requirements are non-negotiable and content scope must be tightly controlled to prevent internal documentation from leaking into customer-visible responses.

The retrieval configuration for the portal applies strict metadata filtering to scope results to customer-facing content only. It weights full-text search more heavily for product and support queries, where terminology precision and tenant isolation matter most.

REMi monitors groundedness and answer relevance continuously across portal queries, providing real-time signals on retrieval quality, without requiring further manual reviews. If groundedness scores dip (indicating the model may be generating answers beyond what the retrieved content supports,) REMi will provide a specific, actionable metric to investigate, rather than a vague report of user dissatisfaction.

Experience 3: Internal Search

Internal search is the use case that most directly replaces legacy enterprise search infrastructure. This is the application employees use to find documents, policies, past decisions, case files and institutional knowledge across the organization's full content repository.

Unlike the employee assistant, which generates natural-language answers, internal search surfaces ranked document results with source attribution and metadata context, allowing users to navigate to the primary source rather than consuming a summarized output.

The retrieval configuration in internal search runs all three index layers simultaneously (semantic for intent-based queries; full-text for exact document retrieval; and knowledge-graph for entity-aware search across related content.) It then returns results ranked by **Reciprocal Rank Fusion** (RRF) across all three.

The same Knowledge Box powers both the employee assistant and the enterprise search tool. Rather than maintaining separate indexes and ingestion pipelines, each application uses its own retrieval configuration to access the same underlying content.

The Architecture That Compounds AI Investments

Enterprise AI programs are stalling because the infrastructure underneath the models responsible for ingestion, indexing, retrieval, evaluation and governance has to be rebuilt from scratch *every time* a new application is deployed. This approach does not scale effectively and increases the operational burden and cost of AI initiatives.

The Agentic Knowledge Layer addresses this challenge by providing a single, continuously updated knowledge foundation. Sync Agents keep content current, while semantic, full-text and knowledge-graph retrieval work together to surface the most relevant information. Teams can tailor retrieval behavior for each application without changing the underlying index or duplicating data. It achieves this through:

- **REMi** continuously evaluating live traffic and providing diagnostic insights when retrieval quality declines
- **Governance** enforced at the retrieval layer through RBAC, tenant isolation, source attribution and compliance controls with every application built on the platform inheriting these capabilities automatically.
- **LLM-agnostic architecture** separating the knowledge infrastructure from the underlying model, allowing organizations to adopt and change models over time without rebuilding the retrieval foundation

This architecture produces compounding returns. The first AI experience you build on the layer establishes the foundation, the second inherits it and subsequent experiences are faster and cheaper to deploy. And since the ingestion pipeline already exists, the index is already populated, the governance controls are already in place and the evaluation framework is already running, each new use case compounds the value of the shared layer rather than adding to the maintenance burden.

The real value of the Agentic Knowledge Layer is not the retrieval technology, but the shared knowledge foundation. Instead of rebuilding knowledge infrastructure for every new AI use case, you invest once in a shared foundation that supports and improves every application built on top of it.

Start with Your Own Data

The architecture described is available to evaluate against your actual enterprise data in a 14-day free trial of the Progress Agentic RAG solution. Connect your content sources, configure a Knowledge Box, run your retrieval strategies and observe REMi quality scores against your own query traffic, not a synthetic benchmark. The hosted trial environment is identical to the production deployment, so what you validate in evaluation is what you ship.

Start your 14-day free trial today

About Progress Software

Progress Software (Nasdaq: PRGS) empowers organizations to achieve transformational success in the face of disruptive change. Our software enables our customers to develop, deploy and manage responsible AI-powered applications and personalized digital experiences with agility and ease. Businesses of all sizes get a trusted provider in Progress, with the products, expertise and vision they need to turn AI disruption into a competitive advantage. Millions of developers and technologists at hundreds of thousands of organizations depend on Progress every day. Learn more at www.progress.com

Worldwide Headquarters

Progress Software Corporation
15 Wayside Rd, Suite 400, Burlington, MA 01803, USA
Tel: +1-800-477-6473

facebook.com/progresssw
x.com/progresssw
youtube.com/progresssw
linkedin.com/company/progress-software
progress_sw_

Progress